

Object Oriented Programming CoSc2212_

June 15, 2025

Compiled by: Natan Asrat

https://www.linkedin.com/in/natan_asrat

Object Oriented Programming

- **OOP:** allow the programmer to break down the problem into objects. It is a programming language model organized around objects rather than "actions" and data rather than "logic"
- **Objects:** self-contained entities consisting of both data and operations together.
- Java:
 - Object oriented
 - Platform independent
 - Simple, Secure, Portable
 - Multithreaded, Robust
- Java platform:
 - Java VM
 - Java API: provides libraries

Object Oriented Programming

- Objects: instance of a class, are runtime entities in OO program
- Class: blueprint for an object, definition of a set of objects
- Members: methods and variables defined in a class
- Abstraction: provide only essential information to outside world

Object Oriented Programming

- Encapsulation: data inside a class is hidden from everything outside the class (can only be accessed by invoking the methods of the class).
- Inheritance: objects of one class get the properties of objects of another class (reusability)
- Polymorphism: ability to take more than one form, a particular operation may behave differently for different sub-classes of the same class

Object Oriented Programming

- Components of java program:
 - Comments:
 - single line `//`,
 - multiline `/* ... */` ,
 - javaDoc `/** ...*/`
 - Token: individual units the compiler recognizes
 - Whitespaces

Object Oriented Programming

- Variables in Java
 - Local variables: declared in methods, constructors or blocks, access modifier can not be used here
 - Instance variables: declared in a class, but outside a block
 - Class/Static variables: one copy of each class variable per class, regardless of how many objects are created from it

Object Oriented Programming

- Data types
 - Primitive (8 types): byte, short, int, long, float, double, char & Boolean
 - Reference: Arrays, classes, and interfaces, String
- Type casting
 - Widening, (Implicit) - Automatic Casting: conversion from one type to another which doesn't lose information
 - Narrowing (Explicit): Java won't do that automatically, do programmatically.

Object Oriented Programming

- Boxing and Unboxing
 - Autoboxing: conversion that the Java compiler makes between the primitive types and their corresponding object wrapper classes. i.e. converting an int to an Integer
 - Boxing: pass the value of the primitive data type to the constructor of the Wrapper class

Object Oriented Programming

- Java statements
 - Expression statements: perform computations and return values
 - Declaration statements: Introduces a very first variables
 - Control flow statements: regulate order of execution (if/else, loops)
 - Jumping Statements: break, continue

Object Oriented Programming

- Types of classes
 - Wrapper Class: eight classes of java.lang package
 - Abstract Class: can't be instantiated.
 - Final Class: can't be inherited.
 - Input-Output Class: contains all the required classes to perform I/O operations ([java.io](#) package)
 - String Class: Sequence of chars
 - Mutable Class: fields that can be changed

Object Oriented Programming

- Types by access modifiers
 - Public Classes: Accessible to the world
 - Private Class: Used only inside a block(e.g with in the outer class)
 - Protected Class: Used in parent class, package and other subclasses
 - Default Class: Used only in parent class and package

Object Oriented Programming

- Method Overloading: process of having the same name for different methods as long as their parameter list or data types in the parameter list is different
- Method Overriding: same name and type signature as a method in its superclass

Object Oriented Programming

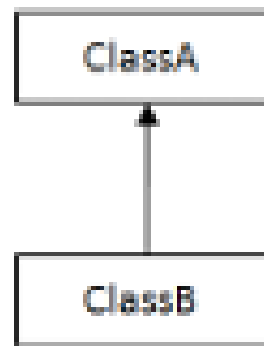
- Constructors: Initializes an object immediately upon creation.
 - Once you define your own constructor, the default constructor is no longer used.
 - Have not return type, not even void
 - Have the same name with their class name
- Garbage Collection: ability of deallocation of memory automatically, when no references to an object exist
 - no explicit need to destroy objects as in C++

Object Oriented Programming

- Access Specifiers:
 - No Modifier(Default): visible to package, aka package-private
 - Private: Visible to the class only
 - Public: Visible to the world
 - Protected: Visible to the package and all subclasses
- Static methods: can access only static data, and static methods.
- Final Variables: prevents the content of that variable being modified
- Final method: prevents from overriding the method

Object Oriented Programming

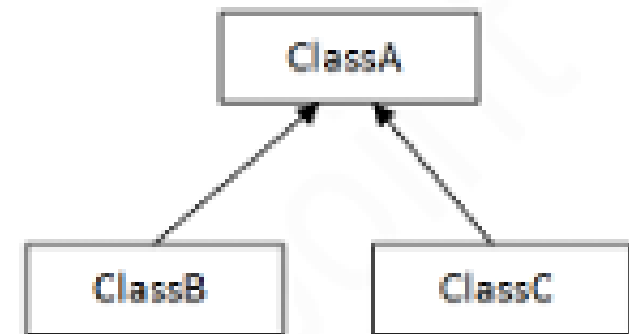
- Types of inheritance:
 - Simple/single inheritance: a class extends another one class only



1) Single



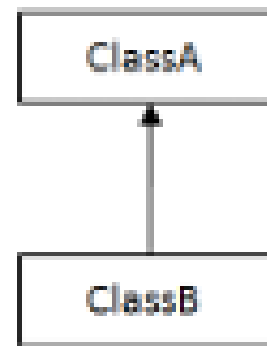
2) Multilevel



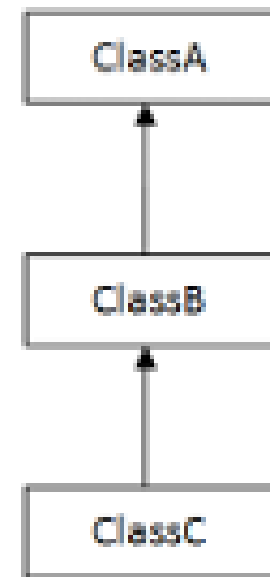
3) Hierarchical

Object Oriented Programming

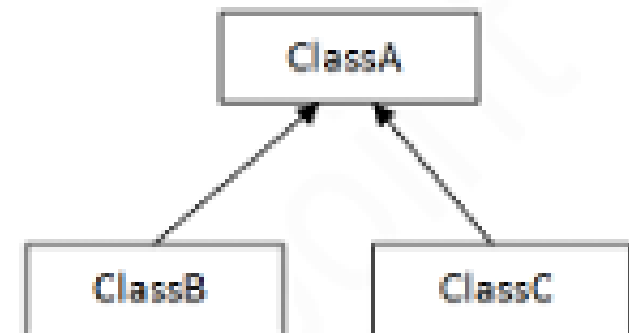
- Types of inheritance:
 - Multi-Level inheritance: inherit from a derived class



1) Single



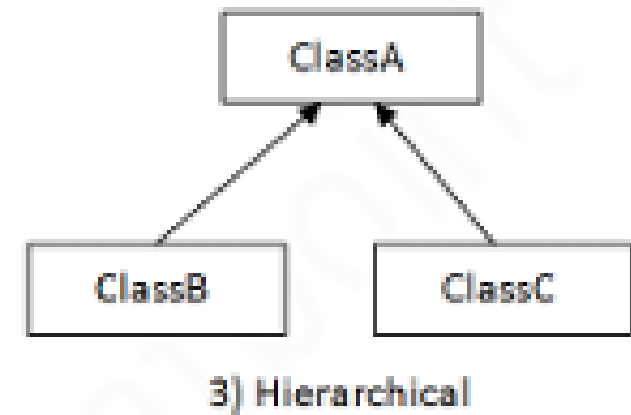
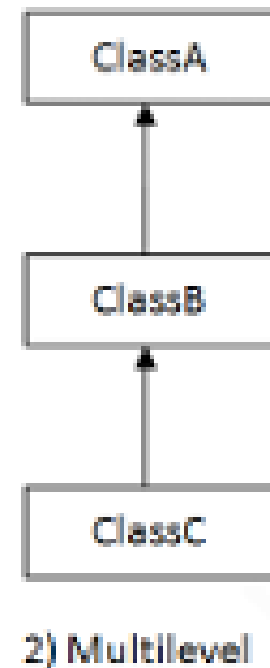
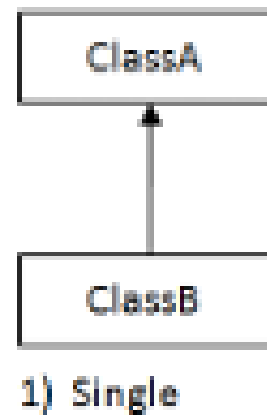
2) Multilevel



3) Hierarchical

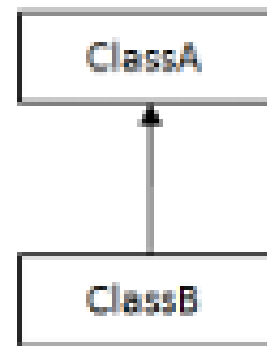
Object Oriented Programming

- Types of inheritance:
 - Hierarchical inheritance: inherited by many sub classes

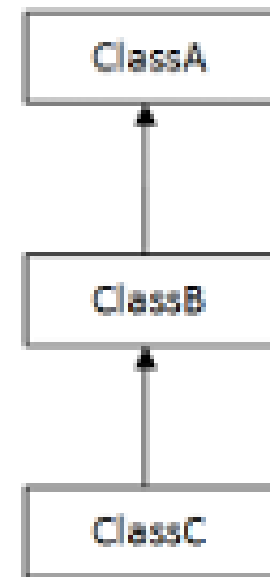


Object Oriented Programming

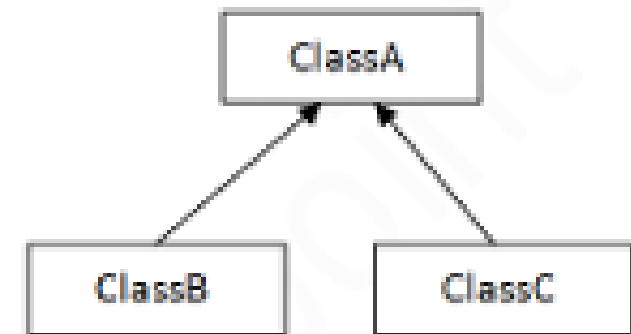
- Types of inheritance:
 - Constructors are not members, so they are not inherited by subclasses



1) Single



2) Multilevel



3) Hierarchical

Object Oriented Programming

- Super
 - Used to access parent class instance members
 - Used to access parent class constructor.
- Rules of method overriding in Java
 - similar argument list
 - exactly matching data types
 - same method name
 - access modifier can not be more restrictive of parent

Object Oriented Programming

- Dynamic Method Dispatch: mechanism by which a call to an overridden method is resolved at run time, rather than compile time (because it is determined by the type of object)
 - Method Overriding is an example of runtime polymorphism
- Abstract Classes: can't be instantiated, cannot declare abstract constructors, or abstract static methods

Object Oriented Programming

- Binding:
 - Late binding: resolves calls to methods dynamically, at run time
 - Early binding: since final methods cannot be overridden, a call to one can be resolved at compile time

Object Oriented Programming

- Types of Polymorphism
 - Static Polymorphism: compile-time binding or early binding - i.e. Method overloading
 - Dynamic polymorphism: late binding

Object Oriented Programming

- Exception: an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions. It is an indication of a problem. aka runtime error
 - A Java exception is an instance of a class derived from Throwable.

Object Oriented Programming

- Exception:
 - Reasons:
 - A user has entered invalid data
 - A file that needs to be opened cannot be found
 - A network connection has been lost
 - JVM run out of memory
 - In general:
 - User error
 - Programmer error
 - Physical resources failed

Object Oriented Programming

- Exception:
 - Types:
 - Checked exception: occurs at the compile time - compile time exceptions.
 - Unchecked exceptions: occurs at the time of execution, - Runtime Exceptions.

Object Oriented Programming

- Exception:

- Types:

- Checked exception: occurs at the compile time - compile time exceptions.
 - Unchecked exceptions: occurs at the time of execution, - Runtime Exceptions.

- Exception Vs Errors

- Errors: serious problem that a reasonable application should not try to catch.
 - Exception: conditions that a reasonable application might try to catch.

Object Oriented Programming

- File: an object on a computer that stores data, information, settings, or commands used with a computer program.

Object Oriented Programming

- Stream: linked to a physical device by the Java I/O system
 - InputStream: to read data from a source
 - OutputStream: for writing data to a destination.
 - Byte Streams: input and output of 8-bit bytes
 - FileInputStream
 - FileOutputStream
 - Character Streams: useful when we want to process text files
 - FileReader
 - FileWriter

Object Oriented Programming

- Java Interface: a blueprint of a class
 - has static constants and abstract methods
 - mechanism to achieve abstraction
 - to support multiple inheritance
 - to achieve loose coupling

Object Oriented Programming

- Rules of Interface
 - Variables declared in an interface are public, static & final by default.
 - can implement more than one interfaces in your class
 - interface extends another interfaces, can extend multiple interfaces
 - has no constructor
 - Methods in an interface are all abstract
- Tagging Interface: empty interface, to claim the membership in a particular set i.e. Serializable, EventListener